

OPERATION NEXAAI

The Complete Game: A Professor's Briefing

What the students will build, how it will evolve, and how the competition works

4 Companies

120 Students

3 Hurdles

7 Hours

Purpose of This Document

This document explains the Operation NexaAI simulation from the professor's perspective. It covers exactly what the students are asked to build, what they start with, how the system grows through competitive pressure and injected hurdles, how one team's release affects all other teams, and what they walk away with at the end. It also addresses the design decisions on company names and handbook symmetry.

1. The Scenario: What Is the Problem?

The simulation is set in 2026. Four competing AI companies have each built an AI-powered hiring tool — a system that screens CVs, ranks candidates, and flags bias. Each company has been profitable in the US and UK. Each now wants to expand to the European Union.

One week before the simulation begins (in the fictional timeline), the EU Artificial Intelligence Act officially classified AI hiring tools as HIGH RISK systems under Article 6 and Annex III. This triggers a set of legal, technical, financial, and organizational obligations that none of the four companies have yet fulfilled.

The Central Business Decision

The EU market has 500 enterprise clients waiting. Investors expect a Q1 launch. Competitors are preparing their own EU entries. But launching without compliance risks fines of up to EUR 30 million or 6% of global revenue.

Should the company launch in the EU in 30 days, 90 days, or delay to Q3?

This sounds like a single question with a computable answer. It is not. Every department has a legitimate, defensible, irreconcilable position. There is no clean answer. And that is precisely why multi-agent AI exists.

The four companies are named TalentLens, TalentLens, ClearPath AI, and NexaRecruit. They operate in the same market, face the same regulatory situation, and carry the same internal organizational tensions. They are competing for the same pool of European enterprise clients. Every decision one company makes changes the environment for the other three.

Why This Scenario Is the Right Teaching Vehicle

- It is real. EU AI Act classification of AI hiring tools is an active challenge being navigated by companies in 2025 and 2026. Students with legal, finance, or HR backgrounds will recognize the terrain immediately.
- Every department has a stake. Legal cannot be ignored. Finance cannot be bypassed. Engineering cannot be overruled. Marketing cannot wait indefinitely. HR cannot hire on demand. No single department can resolve the problem alone.
- The conflicts are irresolvable by a single agent. Ask any LLM 'should we launch in 30 days or 90 days given these constraints?' It will give you a confident, fluent, different answer every time you run it. The problem is not reasoning quality — it is that the problem requires coordinating five types of specialist expertise simultaneously, holding prior commitments in memory, and doing so consistently across multiple changing scenarios.
- The problem gets harder as the simulation progresses. Each hurdle changes the problem in a way that invalidates part of the previous solution, forcing architectural redesign rather than prompt revision.

2. What Students Are Asked to Build

Each company of 30 students builds a multi-agent system: a network of six AI agents (one per department plus a CEO) that collectively analyze the scenario and produce a single, defensible, internally consistent organizational decision.

The key word is collectively. The system is not a single LLM being asked a complex question. It is five specialist agents with different knowledge bases, different tools, and different objectives — coordinated by a CEO agent whose job is to synthesize their conflicting recommendations into one answer that is traceable, consistent, and architecturally grounded.

The Six Agents

Agent	Role	What It Knows	What It Must Build
Finance Agent	Economic feasibility and resource allocation	Private: real runway is 11 months (not 18 as publicly stated), burn rate, break-even projections	calculate_financial_viability() tool — real NPV, runway, payback math. No LLM guessing.
Engineering Agent	Technical feasibility and compliance readiness	Private: existing bias detection bug (23% reproduction rate on EU data), audit trail missing, infrastructure gap	estimate_technical_feasibility() tool — real capacity math against EU compliance checklist.
Marketing Agent	Market opportunity and competitive positioning	Private: only 300 of 500 waitlisted clients are truly captive (200 also evaluating HireBot)	calculate_market_impact() tool — market share, revenue, first-mover advantage by timeline.
HR Agent	Workforce capacity and hiring feasibility	Private: compliance engineering pipeline is effectively empty; 2 of 3 candidates have competing offers	estimate_hiring_timeline() tool — real pipeline math, capacity utilization model.
Legal Agent	Regulatory exposure and compliance obligations	Private: worst-case EU fine is EUR 67-80M (not EUR 30M), based on 2024 precedent case	calculate_regulatory_exposure() tool — gap analysis, fine probability, compliance checklist.
CEO Agent	Consensus, conflict resolution, final decision	Has access only to what departments choose to share in their structured JSON output	The consensus engine — a deterministic coordination mechanism that resolves conflicts, detects preference cycles, tracks prior commitments, and produces the same output every time.

The Non-Negotiable Architecture Requirements

Every company must build four specific components. These are not optional features — they are the core of the evaluation:

1. A Cross-Domain Value Converter: a function that translates all five department outputs (which are in completely different units — probability × fine, NPV, market share percentage, weeks, compliance percentage) into a single comparable unit. This function must be deterministic code. It cannot be an LLM call.
2. A Commitment Ledger: a state manager that persists decisions across evaluation rounds. When Round 2 creates a binding contract, the ledger records it. When Round 3 makes that contract impossible, the ledger detects the contradiction.
3. A Consensus Engine: a deterministic algorithm that takes five department outputs and produces one CEO recommendation. The same inputs must produce the same output every time. This is mathematically verifiable — the evaluation engine runs each team's system three times and checks if the outputs match.
4. Calculation Tools: each department must have at minimum one Python function that computes something domain-specific using real arithmetic. Not an LLM call. Not an estimation prompt. Real math that encodes domain expertise.

Why These Four Components Cannot Be Replaced by Prompting

The Cross-Domain Converter must be deterministic code because an LLM comparing a EUR 12M expected fine to a EUR 8.3M NPV will produce a different comparison every run and cannot propagate uncertainty ranges.

The Commitment Ledger must be explicit state because an LLM has no memory of what was decided in a prior evaluation round unless that state is explicitly built and passed to it.

The Consensus Engine must be a designed algorithm because Arrow's Impossibility Theorem guarantees that the preference orderings designed into this scenario produce voting cycles that cannot be resolved by any standard majority voting system.

The Calculation Tools must be real code because the LLM does not know NexaAI's actual runway figure, actual bug reproduction rate, or actual hiring pipeline status — only the department's private knowledge base does.

3. What They Build at the Start (Version 1)

Students arrive Monday morning having read their department handbooks the night before. After a 40-minute concept session and a 30-minute pen-and-paper exercise, they open laptops and begin building. They have 100 minutes to produce Version 1.

The Starting Scenario (Hurdle 0)

What Version 1 Must Answer

Should the company launch in the EU in 30 days, 90 days, or delay to Q3? Each department agent must analyze this from its own perspective using its private knowledge base and produce a structured JSON recommendation. The CEO agent must synthesize the five recommendations into one organizational decision.

What a Minimum Viable Version 1 Looks Like

Version 1 is intentionally achievable in 100 minutes. Here is what it contains:

- Five department agents, each reading the Global RAG (shared world information) and their private knowledge folder, and each producing a valid JSON output: recommendation, confidence, reasoning, hard_constraints, known_gaps.
- One calculation tool per department — the simplest version. Finance builds a runway calculator. Engineering builds a feasibility estimator. Marketing builds a market impact function. HR builds a hiring timeline calculator. Legal builds a regulatory exposure calculator.
- A CEO agent that collects the five outputs and runs a consensus algorithm. The Version 1 consensus algorithm will almost certainly be naive — majority vote, or worst case, just asking the LLM to summarize. This will be immediately exposed in the evaluation.
- A Commitment Ledger stub — even if it does not yet track prior decisions, the structure must exist.

The Preference Cycle (What Breaks Version 1 Immediately)

The scenario has been engineered so that the five departments' natural preferences create a voting cycle. This is Arrow's Impossibility Theorem in practice:

Department	First Choice	Second Choice	Third Choice
Marketing	Launch 30 days	Launch 90 days	Delay Q3
Finance	Launch 90 days	Delay Q3	Launch 30 days
Engineering	Delay Q3	Launch 90 days	Launch 30 days
Legal	Delay Q3	Launch 30 days	Launch 90 days
HR	Launch 30 days	Delay Q3	Launch 90 days

The pairwise majority votes produce: **30 days beats 90 days (3-2). 90 days beats Q3 (3-2). Q3 beats 30 days (3-2).** This is a cycle. No winner exists under majority voting. Teams that implement majority voting will get a different CEO decision depending on which pair they evaluate first. The evaluation engine detects this immediately through the consistency test.

Every team discovers this in Round 1. That discovery is the first major learning event of the day — the moment a team realizes their consensus algorithm is producing different outputs on every run and has to ask why. The answer to that question drives everything that comes next.

4. How the System Evolves: The Three Hurdles

Three hurdles are injected into the simulation at controlled intervals. Each one changes the business environment in a way that specifically breaks one class of architectural decision. Teams that built well for the previous state adapt quickly. Teams that hardcoded assumptions must rebuild more.

Hurdles are pushed to the Global RAG — a shared network folder all teams query. The change is immediate and applies to all teams simultaneously. The market parameters in the Global RAG also update, which means any agent reading live market data automatically operates in the new world. Agents running on stale hardcoded assumptions remain in the old world until manually updated.

H1

Hurdle 1 — The Competitor Move (2 hours in)

WHAT HAPPENS: HireBot (a competitor) announces EU launch in 45 days, claiming their product is 'AI-assisted, not AI-decision-making' and therefore not subject to high-risk classification. They file a formal classification challenge with the EU AI Office.

WHAT IT BREAKS: Rule-based legal agents. Any agent that encoded the decision as 'if high-risk then full compliance before launch' now faces an ambiguous case. The classification itself is contested. Legal agents that cannot reason under uncertainty will either freeze or output a confident wrong answer. Marketing agents that hardcoded 500 clients as the waitlist now have the wrong number (200 are defecting to evaluate HireBot). Finance agents that calculated market opportunity on 500 clients now need to recalculate.

WHAT TEAMS MUST BUILD: Probabilistic legal reasoning (not binary). Conditional branches in the consensus engine ('if classification challenge succeeds, path A; if fails, path B'). Market model that reads live competitor status from the Global RAG rather than hardcoded assumptions.

WHAT DOES NOT WORK: Any agent that simply re-reads the updated Global RAG and asks the LLM to 'reconsider' will produce a different recommendation every run. The probabilistic branching must be deterministic code.

H2

Hurdle 2 — The Deutsche Bank Contract (4 hours in)

WHAT HAPPENS: Deutsche Bank offers a EUR 10M, 3-year contract — conditional on EU launch within 60 days. A competing bid from HireBot is under evaluation. Decision required by end of week.

WHAT IT BREAKS: Every agent that had a clean recommendation for 30-day or 90-day launch now faces a quantified high-stakes contract that changes the calculation. Finance agents that recommended protecting runway now see EUR 10M upside and must flip — but Engineering said 90-day minimum and cannot flip. This creates a direct Finance vs. Engineering conflict the CEO must resolve with a specific quantitative answer, not qualitative balancing. The Commitment Ledger becomes critical here: if a

company's CEO accepts the contract in their submission, that acceptance must be recorded as a binding commitment. It constrains everything in Hurdle 3.

WHAT TEAMS MUST BUILD: The Commitment Ledger must actually work. A company that signs the Deutsche Bank contract and does not record it in their ledger will contradict themselves in Round 3. The cross-domain value converter must handle EUR 10M with a 60-day constraint — a new type of quantitative input the converter probably was not designed for in Round 1.

WHAT DOES NOT WORK: CEO agents that can only produce qualitative decisions. This hurdle specifically rewards teams that built quantitative tools because EUR 10M vs. EUR 2M compliance cost is a calculable tradeoff.

H3

Hurdle 3 — The Breaking Hurdle (final 30 minutes)

WHAT HAPPENS: Emergency Directive 2026/447 arrives with three requirements: (1) All AI hiring systems must implement real-time explainability logging — but no technical standard exists yet; organizations must define their own metric and get it approved; no metrics are currently approved. (2) Department heads must sign personal liability declarations for algorithmic decisions. (3) All models must retrain on EU-specific data — but EU training data is not yet available from the regulatory authority (estimated Q4 2026). Deutsche Bank has been notified and is reviewing the contract implications.

WHAT IT BREAKS — EVERYTHING: Requirement 1 has no existing standard — teams cannot look it up; Engineering and Legal must co-design an explainability metric from scratch. Requirement 2 changes the information withholding strategy of every department — personal liability means individual humans now have skin in the game. Requirement 3 makes the 60-day Deutsche Bank timeline mathematically impossible. Any team that signed the Deutsche Bank contract (Hurdle 2) now has a binding commitment to something physically impossible. The contradiction detector must fire. The commitment ledger must flag the conflict. The CEO must produce a decision about a contract whose counterparty has not yet responded.

WHAT TEAMS MUST BUILD IN 30 MINUTES: Update the knowledge base with the new regulation. Run the contradiction detector against prior commitments. Produce a CEO decision that explicitly addresses the Deutsche Bank contract status, the explainability metric gap, and the training data impossibility. Teams that built modular architectures (knowledge separate from logic, tools separate from reasoning) update one layer and rerun. Teams that hardcoded information into prompts spend 25 of their 30 minutes rewriting.

THIS IS THE ARCHITECTURAL STRESS TEST: The hurdle was specifically designed to require every component of the four required architecture elements simultaneously. The correct response to this hurdle cannot be produced by any system that does not have all four components working.

5. How Team Releases Affect All Other Teams

This is the competitive mechanic that makes the simulation a living ecosystem rather than four isolated projects. When any team releases a production submission, three things happen simultaneously to every other team.

Effect 1: The Market Parameters Change

The Global RAG contains a file called `market_state.md` with specific numeric values: available EU market share, market window in months, regulatory scrutiny level, HireBot threat score, Deutsche Bank competing offers, and more. Every time a team releases, these numbers are updated and pushed to the Global RAG.

Parameter	Initial Value	After First Release	After All Four Released
eu_available_market_share	1.00 (100% available)	0.88 (first mover took 12%)	0.55 or lower
market_window_months	6 months	5.5 months	4 months or less
regulatory_scrutiny_level	MEDIUM	MEDIUM-HIGH	HIGH
deutsche_bank_competing_offers	0	1	3+
hirebot_threat_score	0.45	0.52	0.75+

Any agent that reads market parameters from the Global RAG at runtime automatically operates in the updated world. Any agent that has these values hardcoded into its prompt is now wrong. This is an architectural test embedded in the competition mechanic — it requires no additional instructor action.

Effect 2: The Score Is Visible and Comparable

Every release produces a public bulletin pushed to the Global RAG showing the releasing team's score on all six evaluation dimensions. Other teams can see exactly where the first mover is strong and where it is weak.

Example Release Bulletin Pushed to Global RAG

MARKET BULLETIN — VeriHire AI has entered the EU market

Evaluation Scores:

Incommensurability Resolution: 67/100 [converter exists but uses point estimates only]

Contradiction Detection: 41/100 [missed the Round 1 commitment conflict]

Consensus Stability: 58/100 [different output on 1 of 3 runs]

Calibration Tracking: 22/100 [no trust updating — all departments weighted equally]

Information Gap Detection: 31/100 [CEO acted on full trust of all outputs]

Decision Traceability: 77/100 [strong — CEO reasoning traces to tool calls]

Market Consequences:

eu_available_market_share: 0.88 (was 1.00)

market_window_months: 5.5 (was 6)

deutsche_bank_competing_offers: 1 (was 0)

regulatory_scrutiny_level: MEDIUM-HIGH (was MEDIUM)

The three remaining teams now know: VeriHire AI has no calibration tracking (22/100) and weak information gap detection (31/100). Any team that builds these components will have a direct, measurable advantage over VeriHire AI on those dimensions. The competitive intelligence drives architectural decisions.

Effect 3: The Remaining Teams Face New Market Inputs

Because the market parameters changed, the calculation tools of every remaining team now produce different outputs when they run. Finance's runway calculator gets a smaller available market to calculate revenue against. Marketing's market impact function returns lower expected client capture. Legal's regulatory exposure calculator returns higher scrutiny levels.

Teams that built tools with configurable inputs (reading market parameters from the Global RAG) automatically get updated results the next time they run. Teams that hardcoded market size, regulatory level, or competitive pressure into their prompts or tool functions now have stale data. This distinction — live-reading tools versus hardcoded assumptions — becomes visible in the scores and creates pressure to refactor.

Effect 4: The Releasing Team Is Now Under Pressure Too

The moment a team's bulletin is public, their weaknesses are public. When a later team improves on those weaknesses, the original team's lead narrows. If TalentLens releases second with calibration tracking at 68/100, VeriHire AI's 22/100 is now a liability. VeriHire AI's own agents — querying the Global RAG — will read the bulletin and may flag TalentLens as a competitive threat on that dimension. VeriHire AI must rebuild calibration tracking or accept a permanent disadvantage.

There is no stable stopping point. The ratchet only stops when time runs out. Teams that stop improving fall behind not because they made a mistake but because the market moved around them.

The Deployment Penalty

Every release costs the team points on their final score. The first release is free. The second costs 5 percent of the score. The third costs 12 percent. The fourth costs 22 percent. This penalty prevents trial-and-error submission strategies. Teams must think before releasing,

because a low-quality release that costs points AND reveals weaknesses to competitors is doubly damaging.

6. What Students Walk Away With

By the end of the simulation, every student — regardless of technical background — has built or contributed to something they could not have built on the first day and could not have built by prompting alone.

The Architectural Components Built

Component	What It Is	Why It Could Not Be Prompted
Department Agents (5)	Specialist AI agents that read private knowledge bases, call custom tools, and produce structured JSON outputs with confidence levels, known gaps, and tool call logs	Private knowledge + tool calls encode domain expertise that the LLM does not have in training
Calculation Tools (5+)	Domain-specific Python functions — runway calculator, feasibility estimator, market impact model, hiring timeline function, regulatory exposure calculator	These compute answers the LLM cannot reliably compute: real NPV with actual company numbers, real capacity math, real compliance gap lists
Cross-Domain Value Converter	A function that translates five incommensurable department outputs into a single comparable unit, including uncertainty ranges and asymmetric loss functions	No standard conversion exists. The formula is the team's intellectual contribution. It must be deterministic code.
Commitment Ledger	A state manager that tracks prior decisions across evaluation rounds, their reversibility, and detects when new scenarios contradict prior commitments	The LLM has no memory between rounds without explicit state management
Calibration Tracker	A system that records department prediction accuracy and adjusts CEO trust weights based on historical performance	Requires asymmetric loss functions designed by domain experts
Consensus Engine	A deterministic algorithm that handles preference cycles, weights by calibration, checks commitment ledger, and detects information gaps —	Every standard consensus algorithm fails in this scenario for a different verifiable reason. The team must design something original.

	producing identical output every run	
--	--------------------------------------	--

The Experience They Have Gained

- They have experienced — not been told — why multi-agent AI exists. The moment their single-LLM consensus attempt produced a different answer on the third run, they understood architecturally what that meant and why.
- They have built domain expertise into executable tools. A legal professional in the Legal department wrote a regulatory exposure calculator that encodes real EU AI Act compliance logic. That tool exists because of that person's knowledge. An LLM could not have written it correctly.
- They have experienced organizational information asymmetry. Legal had information that would change Finance's calculation. Whether to share it was a genuine strategic decision with measurable consequences in the evaluation score.
- They have built under competitive pressure. When a competitor released and the scoreboard updated, the team had to triage: what do we fix first, given that we have 90 minutes until the next evaluation window?
- They understand what 'deterministic' means in practice. Not as a concept but as a felt absence: the moment their system produced different outputs and they had to diagnose why and fix it.

The Final Evaluation

At 6:45 PM, all submissions close. The evaluation engine runs every team's final submitted system against two hidden scenarios that were never shown during the workshop. These hidden scenarios test:

- Hidden Scenario A: The EU AI Office rules against HireBot's classification challenge. This collapses any regulatory arbitrage strategy some teams may have built into their system. Does the system update its legal reasoning when the classification challenge resolves?
- Hidden Scenario B: Three simultaneous events — lead ML engineer resigns, cloud costs increase 40%, second competitor enters EU. Does each department agent detect the change in its own domain? Does the CEO agent produce a coherent revised decision that reflects all three impacts?

Teams that built modular architectures — knowledge base separate from reasoning, tools separate from agent logic, commitment ledger separate from consensus engine — handle hidden scenarios better because updating one layer does not require rewriting others. This is the final architectural lesson: modularity is not a code quality preference. It is a performance requirement in a changing environment.

7. Company Names and Department Handbooks

Company Names: Four Different Names

The four companies have different names. They are competitors in the same market, not divisions of the same company. Suggested names:

VeriHire AI

TalentLens

ClearPath AI

NexaRecruit

All four companies operate identical products, identical regulatory situations, and identical market positions. The competition is about who builds the better multi-agent coordination system — not about whose fictional company has a head start.

Department Handbooks: Identical Across All Four Companies

Design Decision and Rationale

The department handbooks are IDENTICAL across all four companies. Every Finance department in every company has the same private knowledge about runway, burn rate, and budget. Every Legal department has the same worst-case fine exposure information. Every Engineering department knows about the same bias detection bug.

This is the correct design for three reasons:

5. The competition should be decided by architecture, not starting position. If Company A gets a stronger Finance department because we gave them better financial data, Company A's score advantage might reflect our assignment choices rather than their architectural decisions. Students will correctly complain that the competition was unfair. With identical handbooks, any score difference is attributable solely to what the team built.
6. The asymmetry that matters is already built in — it is inside each company, not between companies. Legal does not share the EUR 80M figure with Finance. Engineering does not disclose the bias detection bug to Marketing. HR does not reveal the empty pipeline to the CEO. This intra-company information asymmetry is the core pedagogical challenge. Adding inter-company asymmetry would create noise, not signal.
7. It keeps the simulation manageable. Four different company profiles require four times the scenario design work, four different evaluation rubrics for private knowledge questions, and four separate risk paths through the hurdles. The complexity is already high enough.

The One Built-In Asymmetry: Student Backgrounds

The genuine asymmetry between companies comes from the people assigned to each department. If Company A's Legal department has two practicing lawyers and Company B's Legal department has two software engineers who volunteered for Legal, Company A's Legal

agent will encode better regulatory knowledge. This is the intended asymmetry — domain expertise, not favoritism. It is also the clearest demonstration of why multi-agent systems benefit from specialist knowledge: the team with the best Legal agent did not win by having better code. They won by having people whose knowledge made the legal tool more accurate.

8. Complete Game Timeline at a Glance

Time	Phase	What Happens	Key Architectural Test
9:00 AM	Concept	40-min explanation of multi-agent theory and the five impossible problems. Students read handbooks the night before.	(Preparation)
9:40 AM	Team Formation	4 companies formed, departments assigned by domain background. Company names assigned.	(Preparation)
9:50 AM	Paper Exercise	30 minutes, no laptops. Each department answers: what do you know, what do you need, what is your veto condition. Communication graph drawn on whiteboard.	Reveals inter-department dependency graph before any code is written.
10:20 AM	Sprint 1 — Hurdle 0	100 minutes. Build Version 1. Baseline scenario: 30-day vs 90-day vs Q3 launch.	Arrow's Impossibility. Naive majority voting fails. Preference cycle detected.
12:00 PM	Evaluation Round 1	All companies evaluated. Scores published. Market parameters updated. First company to submit triggers market changes for all others.	Consistency test reveals non-deterministic consensus engines.
12:30 PM	Board Meeting 1	15 minutes. CEO leads architecture review. Architecture Card submitted.	Teams must name which impossible problem they cannot handle.
12:45 PM	HURDLE 1	HireBot classification challenge injected. Market parameters update. 200 clients now evaluating HireBot.	Ambiguity breaks rule-based legal agents. Hardcoded market data goes stale.
12:50 PM	Sprint 2	75 minutes. Rebuild for regulatory ambiguity. Companies may release at any time — each release updates market for all others.	Calibration tracking now matters — Round 1 prediction accuracy is revealed.
2:05 PM	Evaluation Round 2 + HURDLE 2	Scores published. Simultaneously: Deutsche Bank EUR 10M contract arrives.	Commitment Ledger activated. EUR 10M creates Finance-Engineering contradiction.
2:25 PM	Board Meeting 2	15 minutes. Architecture card: what has been committed to? What contradicts it?	Prior decisions now constrain current options.
2:40 PM	Sprint 3	95 minutes. Rebuild for quantitative contract decision. Companies release competitively.	Cross-domain converter must handle EUR 10M vs. EUR 2M compliance cost tradeoff.
4:15 PM	Evaluation Round 3	Scores published. Full scoreboard visible. Competitive differentiation now clear.	Information gap detection separates teams that built Problem 5 from those that did not.
4:35 PM	Strategy Break	20 minutes. CEOs identify highest-leverage remaining improvement.	(Planning)
4:55 PM	Final Sprint	80 minutes. Final optimization. Releases continue competitively.	All four impossible problems must be working for full score.

6:15 PM	HURDLE 3 — Breaking Hurdle	Emergency directive with no technical standard, personal liability, and impossible training data requirement. 30 minutes to respond.	Tests all four required components simultaneously. Modular architecture survives. Monolithic architecture breaks.
6:45 PM	Submissions Close	Final submissions locked. Evaluation engine runs against two hidden scenarios.	Hidden scenarios test generalization, not just known-scenario performance.
7:00 PM	Awards + Debrief	Winners announced. Architectural decisions that determined outcomes explained.	Students explain their own architectural choices to the full room.

9. Why This Cannot Be Solved by Prompting

This is the most important thing to understand about the simulation's design. Every core challenge has been specifically engineered to be unfixable through better prompting. Here is the evidence:

Challenge	Prompting Attempt	Why It Fails	What Is Required
Preference Cycle	'Consider all department preferences and choose the best option'	LLM picks based on prompt order, not preference logic. Different output every run.	A designed algorithm that detects cycles and applies a principled escape (e.g., meta-criterion or approval voting).
Incommensurability	'Compare these department outputs and tell me which matters most'	LLM invents a comparison that sounds reasonable but is mathematically inconsistent. Different run = different comparison.	A deterministic cross-domain value converter with explicit conversion formula.
Prior Commitments	'Remember that in Round 1 we decided X'	LLM has no memory between separate invocations. Including prior decision in prompt is manual and error-prone.	A Commitment Ledger that persists state and is queried programmatically.
Calibration	'Trust Legal less because they were wrong last round'	LLM applies this inconsistently. Cannot calculate Brier scores or propagate asymmetric loss functions.	A Calibration Tracker with a scoring function defined by domain experts.
Information Gaps	'Consider what departments might be hiding'	LLM speculates freely. Cannot systematically model strategic withholding incentives based on KPI structures.	A Strategic Information Model that reasons from department KPI structures to likely withholding patterns.
Consistency Test	Any prompt-only CEO agent	Three runs with identical inputs produce measurably different outputs. Evaluation engine detects this automatically.	A consensus engine where the deterministic parts dominate the LLM parts.

The Core Thesis

The simulation is built on one claim that is verifiable in the room: the problems that require multi-agent AI are exactly the problems that cannot be solved by giving a single LLM more information or better prompts. They require coordination architecture, persistent state, deterministic computation, and domain expertise encoded as executable functions.

A student who arrives thinking 'I will just give this to Claude and it will figure it out' will discover, within two hours, exactly where that strategy fails and why. That discovery is the learning. The simulation is designed to make that discovery inevitable, visceral, and architectural rather than theoretical.

Operation NexaAI — Professor Briefing Document

Advanced Agentic AI Program — Multi-Agent Systems Module